CS 422 Assignment 2

iPerf, TCP Statistics, and Congestion Control

Group member	Name	Purdue ID
1	Kurt Khaustov	kkhausto
2	Kush Nayak	nayakk
3	Nicholas Kann	nkann
4	Aun Ali	ali221
5	Milosz Fecko	mfecko

Repository: github.com/aunali0/CS422_assignment_2. The README lists every command and option; all raw measurements are under `results/`.

1(A) Python iPerf3 Client

Our Python client implements the iperf3 protocol directly, without the iperf3 program: it opens the control connection and exchanges the JSON test parameters, opens the data connection, sends data for the chosen duration, and ends the test the way iperf3 expects (run_test, protocol.py). Timeouts, busy servers ("access denied"), rejected parameters, bad JSON, and dropped connections end the test with a logged error instead of hanging (try_run_test).

1(B) Destination Selection

The client tests `-c` random servers (default 10) from the iperf3serverlist.net list and replaces a failing server with the next untried one, up to `-a` attempts, reporting any shortfall (run_destinations). Each test records its server address and port. For Q3 it took 8 candidate servers to get 3 where all nine tests succeeded; 23 of 72 tests failed, mostly because servers were busy, timed out, or dropped the connection.

1(C) Goodput

Goodput is the increase in the kernel's acknowledged-byte counter (`tcpi_bytes_acked` from the data socket) divided by the actual time since the previous sample (take_sample). Figure 1 and Table 1 use every successful 60-second test of the Q3 experiment, sampled once per second.

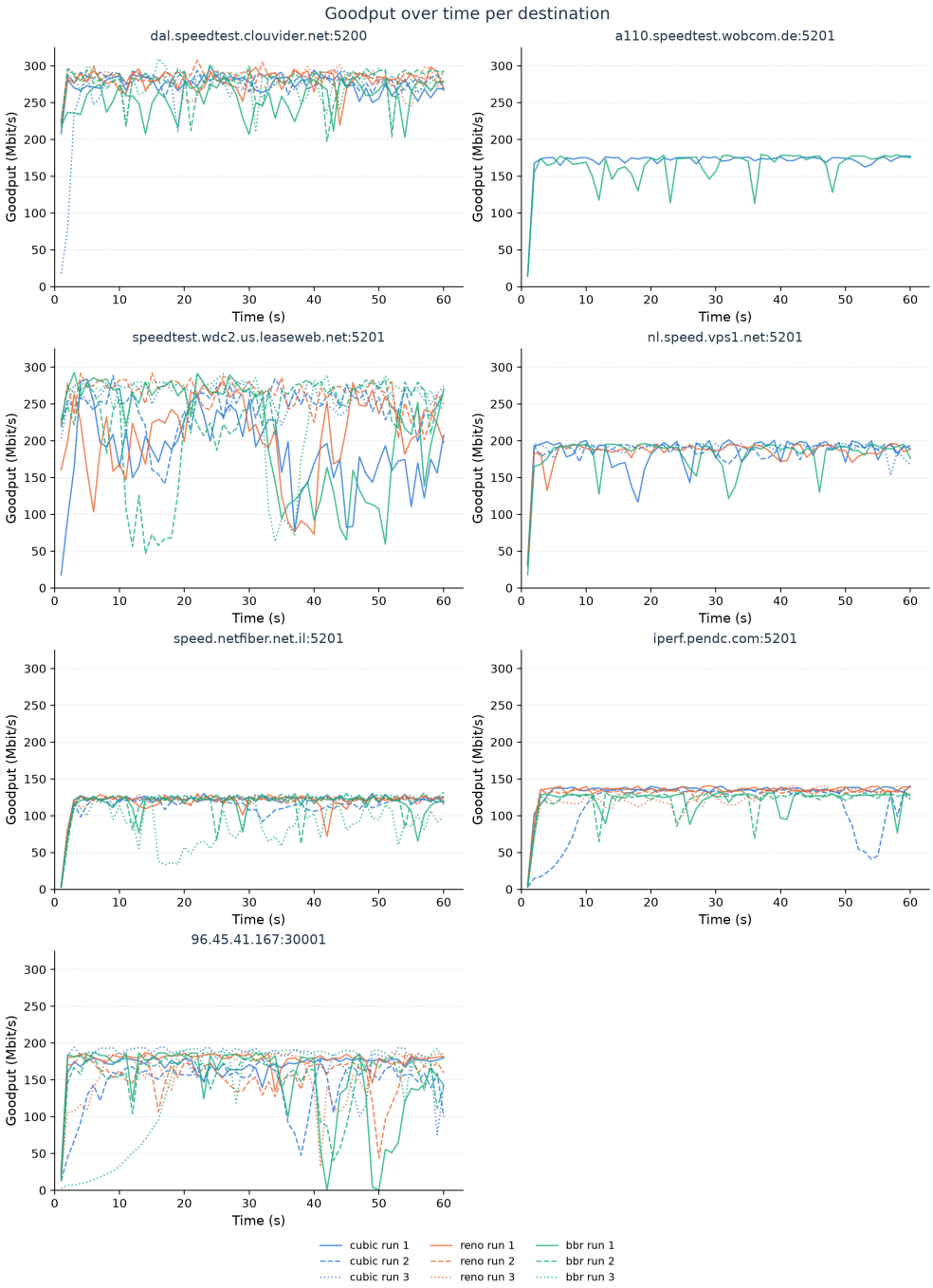


Figure 1. Goodput over time for each destination, one line per run. Color shows the algorithm and line style the run number. Generated by plot_destinations.

Table 1. Goodput in Mbit/s for each destination and algorithm, combining every 1-second sample of all runs. P95 is the 95th percentile (summary_rows).

Destination	Algorithm	Runs	Min	Median	Mean	P95
dal.speedtest.clouvider.net:5200	cubic	3	17.66	279.39	273.96	292.22
dal.speedtest.clouvider.net:5200	reno	3	217.91	283.39	281.30	296.99
dal.speedtest.clouvider.net:5200	bbr	3	197.25	277.92	269.79	295.81
a110.speedtest.wobcom.de:5201	cubic	1	14.70	174.77	170.38	176.24

Destination	Algorithm	Runs	Min	Median	Mean	P95
a110.speedtest.wobcom.de:5201	bbr	1	13.76	173.24	163.87	178.36
speedtest.wdc2.us.leaseweb.net:5201	cubic	3	17.74	248.05	230.50	279.04
speedtest.wdc2.us.leaseweb.net:5201	reno	2	72.87	250.65	235.58	284.81
speedtest.wdc2.us.leaseweb.net:5201	bbr	3	46.73	262.50	229.38	284.11
nl.speed.vps1.net:5201	cubic	3	29.51	191.57	185.64	198.66
nl.speed.vps1.net:5201	reno	1	29.53	187.83	183.82	195.10
nl.speed.vps1.net:5201	bbr	1	18.26	189.43	180.35	195.18
speed.netfiber.net.il:5201	cubic	3	3.59	121.62	118.09	127.95
speed.netfiber.net.il:5201	reno	3	3.59	122.68	119.07	127.38
speed.netfiber.net.il:5201	bbr	3	2.18	119.63	106.59	128.92
iperf.pendc.com:5201	cubic	2	3.05	134.19	121.60	138.71
iperf.pendc.com:5201	reno	3	3.25	133.28	128.15	139.32
iperf.pendc.com:5201	bbr	2	2.85	126.81	120.55	131.10
96.45.41.167:30001	cubic	3	12.57	171.85	163.12	193.20
96.45.41.167:30001	reno	3	16.49	175.33	163.53	185.67
96.45.41.167:30001	bbr	3	0.00	172.38	147.47	189.97

Minimums near zero come from the first second of each run, while TCP is starting up, and from a few short stalls.

2(A) Recording TCP Statistics

Q2 uses CUBIC, the default on our host (kernel `6.18.54.1-microsoft-standard-WSL2`); the algorithm actually used is read back from the socket and saved with every sample. At each goodput sample the client reads `TCP_INFO` from the data socket and saves one CSV row per interval with the time, goodput, `snd_cwnd`, smoothed RTT, `rttvar`, retransmissions, pacing and delivery rates, and the cumulative counters, plus the run ID, destination, algorithm, and units (save_run).

- **Decoding:** the kernel returns Linux's `struct tcp_info` as raw bytes; we unpack its first 248 bytes field by field with Python's `struct` module (tcp_info.py).
- **Interval values:** cumulative counters are differenced between samples; cwnd and RTT are read at the end of each interval.
- **Loss signal:** segments retransmitted per interval (the increase in `tcpi_total_retrans`), an estimate of loss.

2(B) Visualization

Representative destination: `185.180.12.40:5201` (Vienna, about 127 ms RTT), 15 seconds with 0.5-second samples. We chose this run because its loss events show how CUBIC reacts.

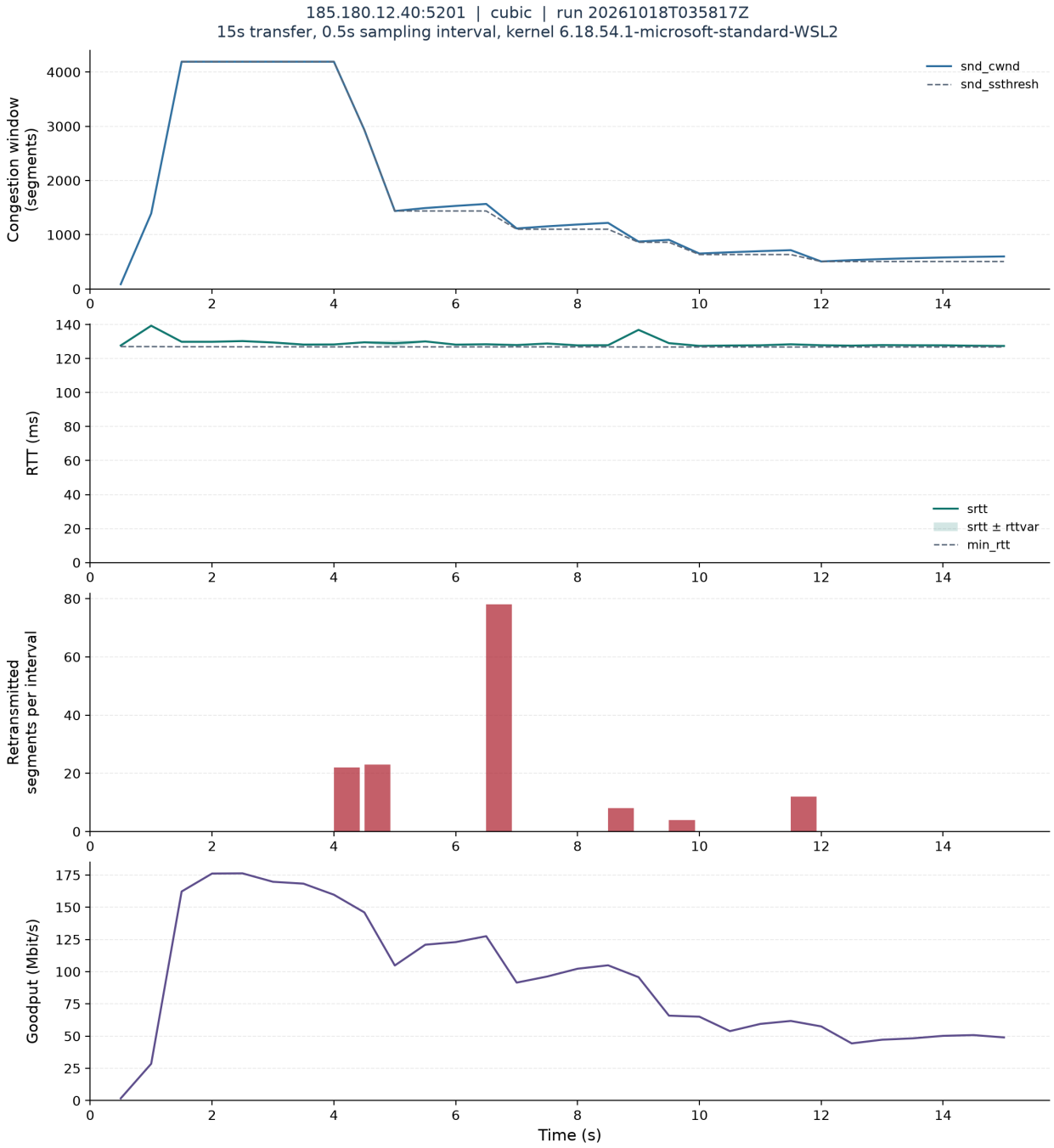


Figure 2. Congestion window and slow-start threshold, RTT (smoothed, ± variation, and minimum), retransmitted segments per interval, and goodput over time (CUBIC). Generated by plot_time_series.

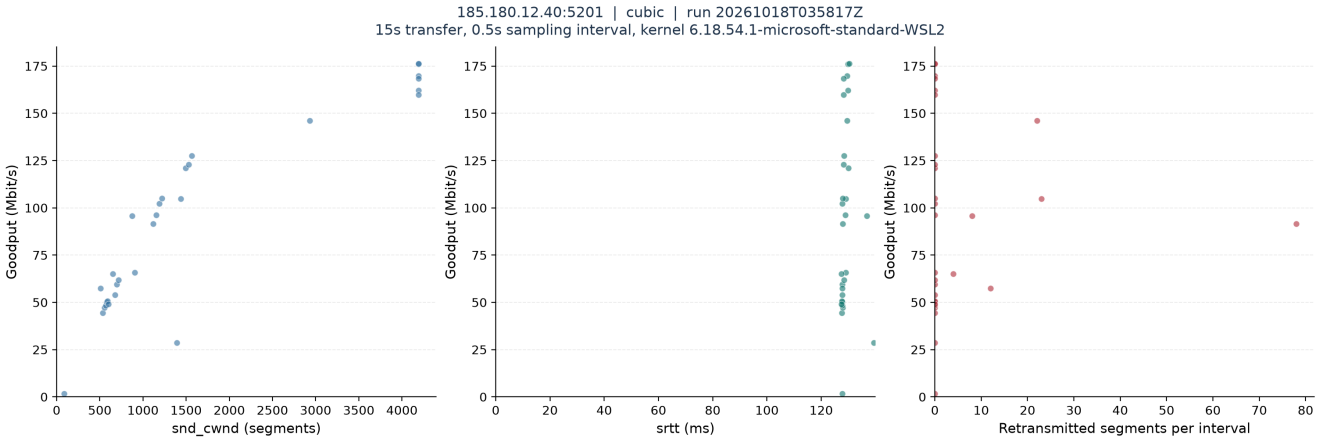


Figure 3. Congestion window, RTT, and retransmissions per interval, each plotted against the goodput of the same interval. Generated by plot_scatter.

2(C) Observations

What each metric means

- **Congestion window:** how many segments TCP allows to be unacknowledged at once. When it is the limit, goodput ≈ window × segment size ÷ RTT, so a bigger window means more goodput. In congestion avoidance CUBIC grows the window slowly and cuts it by 30% after a loss [2].
- **RTT:** how long data takes to be acknowledged. The lowest RTT is roughly the path's travel time; anything above it is time spent waiting in queues. With the same window, a higher RTT means lower goodput.
- **Loss signal (retransmissions):** after a loss CUBIC shrinks its window, so goodput should drop and then recover slowly.
- **Goodput:** how fast our data is actually acknowledged. It is the result of the other three.

What we saw

Start-up: The window grew from 88 to 4,191 segments in the first 1.5 seconds. Slow start then ended without any loss (the slow-start threshold was set equal to the window). This matches Linux CUBIC's HyStart, which leaves slow start as soon as RTT starts to rise.

The window was not the limit at first: From 1.5 to 4 seconds the window allowed about 355 Mbit/s, but goodput was only 160–176 Mbit/s. Only about 2,000 segments were actually in flight and almost nothing was waiting to be sent, so our sender, not the network, was the bottleneck. Linux does not grow a window that is not being used, which is why it stayed flat.

Losses cut the window: Retransmissions at 4.5, 5, 7, 9, 10, and 12 seconds each cut the window to about 70% of its size (for example 1,568 → 1,102 and 906 → 635 segments).

After that, goodput followed the window: From 5 seconds on, the window was full and goodput matched window × segment size ÷ RTT within a few Mbit/s (121 vs 125, 62 vs 61, 48 vs 49 Mbit/s). That is why the window–goodput scatter in Figure 3 is almost a straight line.

Anomaly, losses without extra delay: RTT stayed at 127–130 ms, within 3 ms of its minimum, for the whole test, so the RTT–goodput scatter is a vertical band. The losses therefore did not come from a queue filling up, which would have raised RTT; a small router buffer, a rate limiter, or random loss on the path would look like this. CUBIC still treated each loss as congestion, and goodput fell from 176 to about 50 Mbit/s.

Anomaly, slow recovery: Between losses the window grew by only about 14 segments per round trip, so 15 seconds was not long enough to get back to the earlier rate.

3(A) Compare CUBIC, Reno, and BBR

We compared CUBIC, Reno, and BBR at three destinations, using three 60-second runs per algorithm and one-second sampling: 27 matched runs. Each algorithm was set on the data socket before connecting, read back, and recorded (data_connection). The host ran Linux `6.18.33.2-microsoft-standard-WSL2` and Python 3.14.8. Its recorded library versions were matplotlib 3.11.2 and pydantic 2.13.5.

Destinations were shuffled with seed 422. Algorithm order rotated across repetitions, placing each algorithm first, second, and third once per destination (run_comparison). Dallas used BBR/CUBIC/Reno, CUBIC/Reno/BBR, then Reno/BBR/CUBIC; Netfiber began with the third of these orders and the IP destination with the second. Tests ran sequentially with a two-second pause. Dallas was the first complete group and became the representative without selecting by throughput. Five incomplete destination groups were excluded; their partial measurements and all failures remain in the logs.

Goodput uses changes in acknowledged bytes divided by actual elapsed time. Algorithm means average three duration-weighted run means; ± values are sample standard deviations across runs. RTT is the mean sampled smoothed RTT. Retransmission totals are a loss proxy, not packet-loss probabilities (run_statistics). The CSV summaries additionally provide per-run minimum, median, arithmetic mean, and linearly interpolated P95 goodput.

Table 2. Mean results across three repetitions. Goodput ± values show sample standard deviation across runs; retransmissions are totals across the three runs.

Destination	Algorithm	Goodput (Mbit/s)	Mean RTT (ms)	Retrans. (segs)
Dallas	CUBIC	273.96 ± 5.63	68.73	224
Dallas	Reno	281.30 ± 2.76	73.65	444
Dallas	BBR	269.79 ± 11.69	64.53	196
Netfiber	CUBIC	118.08 ± 4.16	181.90	23
Netfiber	Reno	119.07 ± 0.89	181.67	41
Netfiber	BBR	106.59 ± 17.37	177.39	398
IP destination	CUBIC	163.12 ± 17.51	132.58	120
IP destination	Reno	163.52 ± 11.80	136.69	11
IP destination	BBR	147.47 ± 6.02	150.60	541

Dallas: `dal.speedtest.clouvider.net:5200`. Netfiber: `speed.netfiber.net.il:5201`. IP destination: `96.45.41.167:30001`.

Observations

Dallas: Reno delivered 281.30 Mbit/s, CUBIC 273.96, and BBR 269.79. BBR had the lowest mean RTT (64.53 ms), while Reno had the highest (73.65 ms). Thus Reno's modest throughput advantage came with higher measured delay. Reno's median cwnd rose from 1,982 to 7,854 segments across repetitions while throughput stayed near 279–284 Mbit/s. A larger window budget did not yield proportionally more delivery. Minimum filtered RTT was about 29–32 ms; higher observed RTT is consistent with queueing or other variable delay, including reverse-path delay.

Netfiber: With minimum RTT around 170–174 ms, Reno and CUBIC achieved similar throughput (119.07 versus 118.08 Mbit/s). Their difference was small relative to CUBIC's run variation. BBR averaged 106.59 Mbit/s and slightly lower RTT. Its third run delivered only 86.57 Mbit/s with 396 retransmissions, compared with 115.56 and 117.63 Mbit/s in its first two runs. RTT stayed close to baseline during much of the slowdown, showing why a loss

signal is useful alongside RTT.

IP destination: Reno and CUBIC were effectively tied (163.52 versus 163.12 Mbit/s), with run standard deviations of 11.80 and 17.51 Mbit/s. BBR averaged 147.47 Mbit/s and had more variable RTT. In its first run, the interval at about 50 seconds had zero acknowledged goodput and a 1,559.64 ms RTT, with retransmissions increasing. The transfer still completed. This stall contributes to the averages; it was not removed as an outlier.

Across these paths, Reno had the highest mean throughput, BBR had lower mean RTT on two paths, and neither CUBIC nor BBR established a throughput win. These are observed operating regimes, not universal rankings. Reno's additive growth [1], CUBIC's faster growth on suitable large-bandwidth-delay paths [2], and BBR's bandwidth/RTT model [3] explain their intended behavior, but do not prove what caused differences between sequential Internet tests.

For a window-limited flow, rate is approximately cwnd × MSS / RTT. In Dallas's third Reno run, median window budget was about 11.37 MB, versus a 1.09 MB achieved-rate × minimum-RTT proxy. The proxy is not an independent bottleneck-capacity estimate, and cwnd is not actual bytes in flight. Server limits, receiver windows, pacing, or path capacity can limit delivery.

Public routes and cross-traffic changed between tests; rotating order cannot remove that variation. Three runs and one-second samples also limit conclusions. These single-flow tests do not establish fairness.

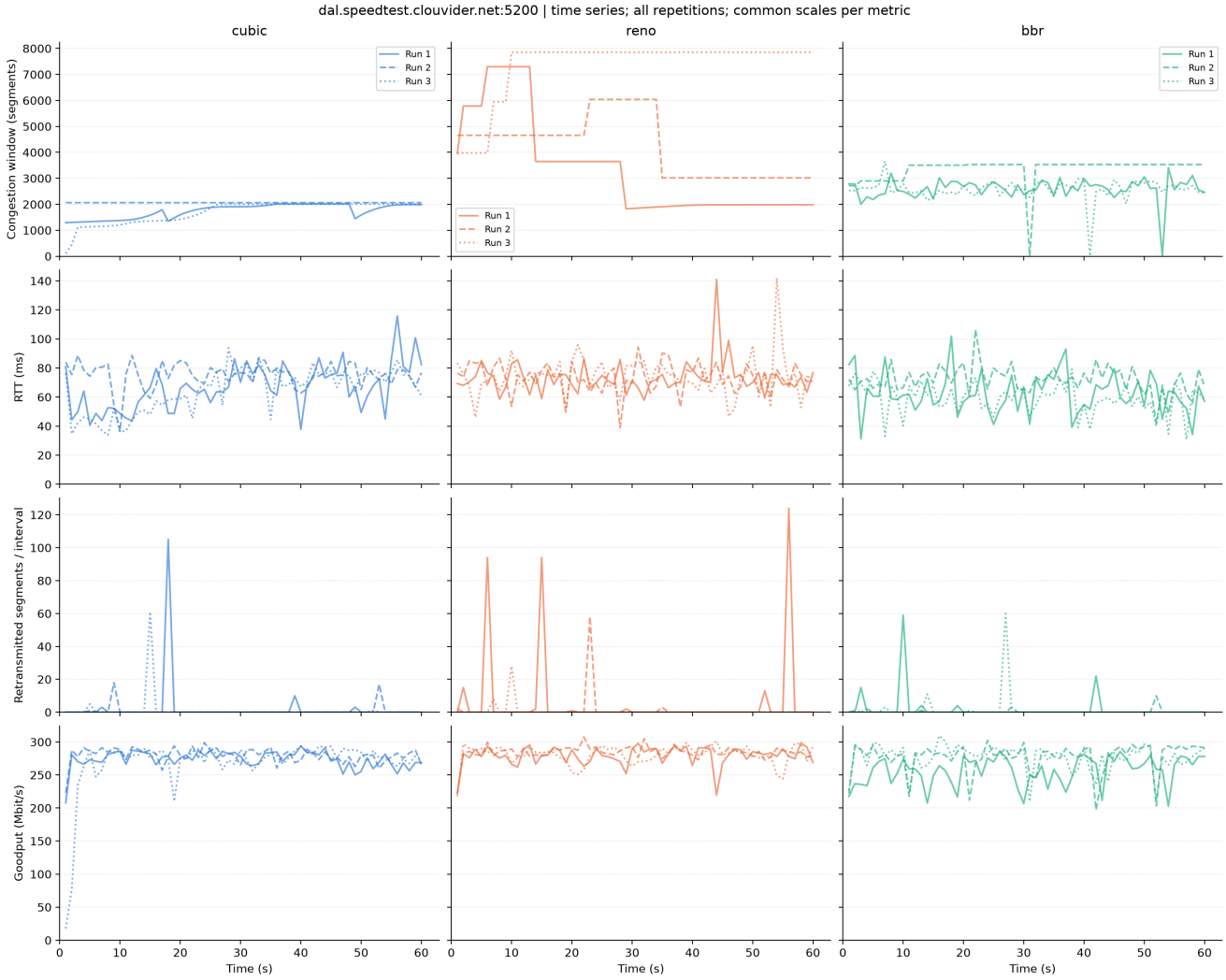


Figure 4. Congestion window, smoothed RTT, retransmissions, and goodput over time at the representative Dallas destination. All repetitions are shown with common scales across algorithms. Generated by plot_comparison.

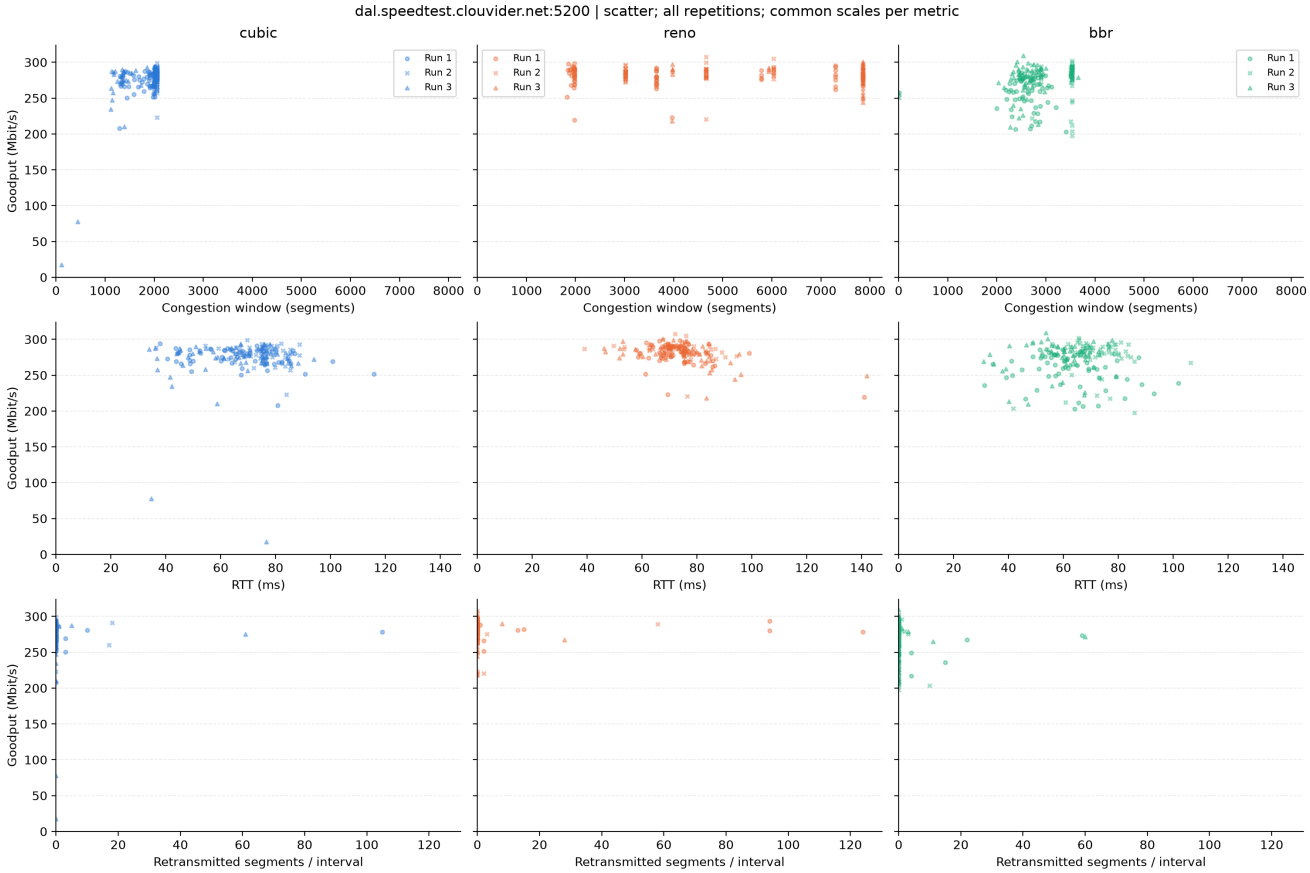


Figure 5. Congestion window, RTT, and retransmissions against goodput at the same representative destination. Markers distinguish repetitions.

3(B) Proposed Congestion Control Algorithm

The proposed controller uses smoothed RTT, acknowledged goodput, and new loss-recovery events. Start with cwnd = 10 segments and ssthresh = infinity; minimum cwnd is 2. Update once per RTT after ACK progress. Let base be the minimum valid RTT over 10 seconds. Measure goodput per update (epoch) and smooth it with rate = $0.75 \times$ old_rate + $0.25 \times$ epoch_goodput, initialized from the first valid epoch.

```
State: cwnd = 10; ssthresh = infinity; slow_start = true
On entering a new recovery episode, once per episode:
    ssthresh = max(2, floor(cwnd / 2)); Linux applies reduction
    slow_start = false; reset epoch accounting
On retransmission timeout, through the TCP stack:
    ssthresh = max(2, floor(cwnd / 2)); cwnd = 1
    slow_start = true; reset epoch accounting
Once per RTT with new ACKs, outside recovery:
    update base (minimum RTT) and rate (epoch goodput)
    if app limited or receiver-window limited: reset; finish
    if slow_start and cwnd < ssthresh and srtt <= 1.10 * base:
        cwnd += min(newly_acked_segments, cwnd); finish
    slow_start = false
    if srtt > 1.25 * base and rate <= old_rate:
        cwnd = max(2, floor(0.90 * cwnd)); ssthresh = cwnd
    else if srtt <= 1.10 * base:
        cwnd += 1
    otherwise hold; enforce kernel cwnd clamp
```

The Dallas window/rate mismatch motivates the throughput gate; Netfiber motivates a separate loss response; the IP-path stall motivates responding to delay inflation. The hold band between 10% and 25% RTT inflation and the goodput smoothing aim to reduce oscillation. The thresholds are hypotheses, not tuned or validated results. `Q3_DESIGN.md` discusses the design in more detail.

Assuming persistent flows, equal RTTs, and a shared bottleneck, additive growth and proportional reductions may encourage similar windows. Fairness and convergence are not guaranteed: unequal RTTs favor different update rates, delay-sensitive flows can yield to loss-based flows, and stale RTT/rate estimates can preserve unfair allocations. Repeated increases and decreases may produce a limit cycle rather than a fixed rate. Assignment 3 must implement and evaluate the proposal; these baseline measurements do not evaluate it.

Reproduction

Everything runs from `main.py`; the README documents the commands and options (`-c` destinations, `-t` duration, `-g` sampling interval, `-r` repetitions), and the `Dockerfile` (`FROM ubuntu:24.04`) runs the same pipeline. Hosts: Q2 kernel `6.18.54.1-microsoft-standard-WSL2`, Q3 kernel `6.18.33.2-microsoft-standard-WSL2`. BBR must be available on the host (`sudo modprobe tcp_bbr`), and the container needs `--cap-add NET_ADMIN` to select algorithms outside `net.ipv4.tcp_allowed_congestion_control`.

References and AI Acknowledgement

[1] RFC 5681: TCP Congestion Control: rfc-editor.org/rfc/rfc5681

[2] RFC 9438: CUBIC: rfc-editor.org/rfc/rfc9438

[3] Google BBR documentation: github.com/google/bbr

AI assistance was used for automation, plotting, checking, and drafting the analysis and design. A teammate collected the measurements on Linux/WSL2.